RAKE PLANNING SYSTEM

A Dissertation submitted in partial fulfilment of the requirements for the award of degree of

MASTER OF COMPUTER APPLICATION

Of

CHHATTISGARH SWAMI VIVEKANAND TECHNICAL UNIVERSITY, BHILAI, INDIA

By

GIRDHR AGRAWAL

Enrolment Number: CF1330

Under the Guidance of

PROF. DEEPAK PANDEY

DEPARTMENT OF MASTER OF COMPUTER APPLICATION,
RUNGTA COLLEGE OF ENGINEERING & TECHNOLOGY,
KOHKA-KURUD ROAD, BHILAI - 490024,
CHHATTISGARH, INDIA

Session: 2025-26

RAKE PLANNING SYSTEM

A Dissertation submitted in partial fulfilment of the requirements for the award of degree of

MASTER OF COMPUTER APPLICATION

Of

CHHATTISGARH SWAMI VIVEKANAND TECHNICAL UNIVERSITY, BHILAI, INDIA

By

GIRDHR AGRAWAL

Enrolment Number: CF1330

Under the Guidance of

PROF. DEEPAK PANDEY

DEPARTMENT OF MASTER OF COMPUTER APPLICATION,
RUNGTA COLLEGE OF ENGINEERING & TECHNOLOGY,
KOHKA-KURUD ROAD, BHILAI - 490024,
CHHATTISGARH, INDIA

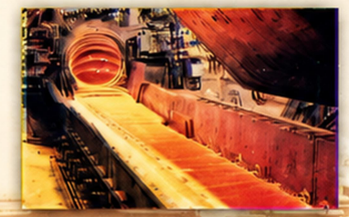
Session: 2025-26

सेल SAIL

भिलाई इस्पात संयंत्र
BHILAI STEEL PLANT

मानव संसाधन विकास विभाग
HUMAN RESOURCES DEVELOPMENT DEPARTMENT

प्रमाणपत्र
CERTIFICATE



पंजीयन क्र.
Regn. No. P-25/5707

प्रमाणित किया जाता है कि श्री / कुमारी
This is to certify that Shri / Ku. *Girdhar Agrawal*

सेमेस्टर विद्यार्थी
Third Sem., student of MCA of RCET, Bhilai कालेज / संस्थान College / Institute

ने अवकाश कालीन प्रशिक्षु के रूप में दिनांक से तक प्रशिक्षण प्राप्त किया ।
has undergone project based training from *22/12/2025* to *14/03/2026*

प्रोजेक्ट
Project *Report on" Rate Planning System"*

इस अवधि में उनका कार्य निष्पादन रहा ।
His / her performance during ths training period has been *Excellent*

14/3/26

प्रभारी (प्रशिक्षण)
Incharge (Training)

भिलाई, दिनांक
Bhilai, Dated *14/03/2026*

सुष्मिता पाल / Sushmita Paila
उ.प्र.(मा.सं.-डी.एम.एच.) / DM (HR-Trg.)
SAIL (सेल) BSP भिलाई इस्पात संयंत्र

PROJECT COMPLETION CERTIFICATE FROM BHILAI STEEL PLANT

DECLARATION

I, **Girdhar Agrawal** a student of 4th Semester MCA, Rungta College of Engineering & Technology, Bhilai, bearing Enrolment Number **CF1330** hereby declare that the project entitled "**Rake Planning System"** has been carried out by me under the supervision of External Guide **J. P. S. Chauhan**, **General Manager (C & IT) & Gurmeet Singh, General Manager (Plate Mill),** at **Bhilai Steel Plant.** The project is being submitted in partial fulfilment of the requirements for the award of the Degree of Master of Computer Application of the Chhattisgarh Swami Vivekanand Technical University, Bhilai, India during the academic year 2025-26. This report has not been submitted to any other Organization/University for any award of degree.

Signature :

Student Name : Girdhar Agrawal

University Roll No. : 501302124014

Date : / / 2026

FORWARDING CERTIFICATE

This is to certify that **Girdhar Agrawal,** a bonafide student of Master of Computer Application at Rungta College of Engineering & Technology, Bhilai, has carried out his project work as mentioned in this project entitled "**Rake Planning System"** at **"Bhilai Steel Plant",** during his fourth semester of studies in MCA as a part of a curriculum for obtaining the degree of MCA of the Chhattisgarh Swami Vivekanand Technical University, Bhilai, Chhattisgarh, India to which the institute is affiliated.

This certificate issued by the undersigned does not cover any responsibility regarding the statements made and work carried out by the concerned student.

The current dissertation is hereby being forwarded for evaluation for the purpose for which it has been submitted.

Prof. Deepak Pandey

Project Coordinator (Department of MCA)

RCET, Bhilai

Date : / / 2026

Prof. Dr. Ajay Kushwaha

Head, Department of MCA,

RCET, Bhilai

Date : / / 2026

CERTIFICATE OF APPROVAL

This is to certify that the project entitled "**Rake Planning System",** carried out by **Girdhar Agrawal,** students of 4th semester, MCA at Rungta College of Engineering & Technology, Bhilai**,** is hereby approved after proper examination and evaluation as a creditable work for the partial fulfilment of the requirements for awarding the degree of Master of Computer Application of the Chhattisgarh Swami Vivekanand Technical University, Bhilai, Chhattisgarh, India.

(Internal Examiner)
Name :
Designation :
College Name : RCET, Bhilai
Date : / / 2026

(External Examiner)
Name :
Designation :
College Name :
Date : / / 2026

ACKNOWLEGEMENTS

I have great pleasure in the submission of this project report entitled "**Rake Planning System**" for **"Bhilai Steel Plant"** in partial fulfilment of the degree of Master of Computer Application of the Chhattisgarh Swami Vivekananda Technical University, Bhilai, Chhattisgarh, India. While submitting this project report, I take this opportunity to thank all those who are directly or indirectly related to this project work.

I would like to thank my guide **J. P. S. Chauhan & Gurmeet Singh** who has provided me the opportunity and organized a project for me. Without his active cooperation and guidance, it would have become very difficult for me to complete the work in time.

I would like to express my deepest gratitude to my parents and the management of Rungta College of Engineering and Technology, Bhilai, honourable **Shri Santosh Ji Rungta**, Chairman, honourable **Prof. Dr. Sourabh Rungta**, Vice Chairman, and honourable **Shri Sonal Rungta,** Secretary for their continuous moral support and encouragement.

I would like to express my sincere thanks to respected **Prof. Dr. Y. M. Gupta Sir**, Director Academics, Rungta College of Engineering & Technology, Bhilai**,** respected **Prof. Dr. Chinmay Chandrakar Sir**, Dean Academics, Rungta College of Engineering & Technology, Bhilai**,** respected **Prof. Dr. Ajay Kushwaha Sir,** Head, Department of MCA, Rungta College of Engineering & Technology, Bhilai, and respected **Prof. Deepak Pandey Sir**, Project Coordinator, Department of MCA, Rungta College of Engineering & Technology, Bhilai.

Last but not the least, I also thank the **faculty members** and **staff members** of Rungta College of Engineering & Technology, Bhilai**,** for their continuous help and guidance throughout the work of project. Acknowledgements are due to my family members, friends, and all those persons who have helped me directly or indirectly in the successful completion of the project work.

Name of the Student : Girdhar Agrawal
University Roll No. : 501302124014
Enrolment No. : CF1330

CONTENTS

CHAPTER 1 : INTRODUCTION

1.1 INTRODUCTION

The steel industry demands precise logistics management, especially in the dispatch of finished steel products to customers spread across different geographic regions. Rake planning — the process of grouping and scheduling wagon loads into a full rake (a fixed set of wagons) for rail dispatch — is one of the most critical, data-intensive operations in any integrated steel plant.

Bhilai Steel Plant (BSP), one of the premier steel producers in India under SAIL (Steel Authority of India Limited), handles a very large volume of steel plate orders meant for various consignees across the country. Managing these orders, calculating the number of wagons required, and planning rake dispatches has traditionally been a manual or semi-automated task involving spreadsheets and on-premise tools.

The BSP Rake Planning System is a modern, full-stack web application designed to digitize and automate this process. It provides a unified platform where authorized users can upload order data, apply multi-dimensional filters, visualize key performance indicators (KPIs), and plan rake dispatch operations in real time. The system replaces cumbersome spreadsheet-based workflows with a responsive, browser-accessible dashboard backed by robust data-processing logic.

1.2 PROJECT DESCRIPTION

The BSP Rake Planning System is a full-stack web application developed to automate and simplify the rake planning process at Steel Authority of India Limited Bhilai Steel Plant. The system helps manage steel plate dispatch operations by providing a centralized platform for uploading, processing, filtering, and analyzing order data in real time. It replaces manual spreadsheet-based workflows with an efficient browser-based dashboard that improves accuracy, speed, and decision-making.

The application supports data ingestion from CSV/Excel files and REST APIs, performs data cleansing and normalization, and provides advanced filtering based on destination, quality, HT type, region, and dispatch mode. It also offers KPI dashboards, consignee summaries, thickness matrix analysis, graphical reports, and CSV export functionality. A separate GM module is included for grade mix comparison analysis. The system uses JWT-based authentication with role-based access control for Admin, Pmgm, Citgm, and Viewer users.

MAIN FEATURES

- Upload and process CSV/Excel order data
- Fetch live data from REST APIs
- Multi-filter dashboard for order analysis
- Wagon and rake calculation system
- KPI dashboards and graphical reports
- Consignee and thickness-wise analysis
- CSV export functionality
- GM comparison module
- JWT-based authentication and role management

The system has following **Users** and **Roles**:

Role	Description
Admin	Full access to all modules and system settings
Pmgm	Standard operational access
Citgm	Department-specific operational access
Viewer	Read-only dashboard acces
Authentication	JWT-based role-aware login with multiple user roles

1.3. COMPANY PROFILE

1.3.1 About Bhilai Steel Plant (BSP)

Bhilai Steel Plant is one of India's largest integrated steel plants and operates under Steel Authority of India Limited, a Government of India enterprise under the Ministry of Steel. Established in 1959 with the collaboration of the former U.S.S.R., BSP is located in Bhilai, Chhattisgarh, and produces rails, structural steel, wire rods, plates, blooms, and billets. The plant is a major supplier to Indian Railways and is known for producing long railway rails and high-quality steel products.

1.3.2 Plate Mill and Dispatch Operations

The Plate Mill at BSP manufactures steel plates of different sizes and grades such as Mild Steel, High Tensile, and Boiler Quality plates. Finished products are dispatched mainly through railway rakes, where each rake consists of multiple wagons carrying steel materials. The dispatch process involves coordination between production, sales, and logistics departments to ensure efficient rake planning and timely delivery of orders.

CHAPTER 2 : SYSTEM STUDY

2.1 EXISTING AND PROPOSED SYSTEM

This is an important phase in software development that involves understanding the existing system, identifying its limitations, and proposing an improved solution.

2.1.1 EXISTING SYSTEM

Before this system was developed at BSP's plate dispatch section was handled through:

- Manual Power Bi system maintained by multiple individuals.
- Periodic email communication for data sharing across departments.
- Low **Access Control** - No Shared Data.
- **Availability** only on local machines
- Also lack a unified, role-aware, web-based tool and limited visibility for management.

2.1.2 PROPOSED SYSTEM

- Full-stack web application developed using FastAPI for the backend and React for the frontend.
- Supports CSV/Excel file uploads and REST API data fetching with 24-hour caching.
- Browser-based system accessible from any device within the network or internet.
- Provides multi-select real-time filters with pre-configured filter presets.
- Provides multi-select real-time filters with pre-configured filter presets.
- Automated calculations performed using Python libraries such as Pandas and NumPy.
- Interactive dashboards and charts using Recharts with multiple chart types.
- JWT-based authentication with role-based access management.
- Supports cloud deployment on platforms like Render, AWS, and Docker.

ADVANTAGES OF THE PROPOSED SYSTEM

- Centralized platform providing a single source of data.
- Secure role-based access control for users.
- Automated real-time calculations reduce manual work.
- API caching minimizes dependency on repeated manual uploads.
- Supports remote accessibility and cloud deployment.
- Faster and more efficient rake planning process.
- Improved data accuracy and operational efficiency.

2.2 FEASIBILITY STUDY

Feasibility study determines whether the proposed system is practical and viable. The feasibility of the Rake Planning System is analyzed under the following aspects:

2.2.1 TECHNICAL FEASIBILITY

The project uses well-established, open-source technologies:

- **Python** / **FastAPI** — mature, high-performance web framework.
- **React** + **TypeScript** — industry-standard frontend stack.
- **Pandas** / **NumPy** — proven data processing libraries.
- **Docker** — standard containerization platform for deployment.

All technologies are freely available and well-documented. The development team has the required skills. The system can be hosted on commodity cloud infrastructure (Render.com, AWS EC2/ECS, or any Linux server). → **Technically Feasible**.

2.2.2 ECONOMIC FEASIBILITY

- All software components are open-source (zero licensing cost).
- Hosting on Render.com free/starter tier or AWS has a predictable low monthly cost.
- Eliminated manual effort equivalates to several person-hours saved per day.
- Reduction in dispatch errors translates into direct cost savings. → **Economically Feasible.**

2.2.3 OPERATIONAL FEASIBILITY

- The system requires only a modern web browser — no client-side installation.
- The interface is designed to mirror the mental model of existing Excel users — minimal retraining needed.
- The role-based system means users only see relevant panels, reducing cognitive overload.
- Data can be uploaded via CSV (familiar format) or fetched automatically. → **Operationally Feasible**.

2.3 TOOLS AND TECHNOLOGIES USED

The following tools and technologies are used in the development of the Rake Planning System.

BACKEND TECHNOLOGIES

- Python, Fast API (Core Programming and REST API framework).
- Pandas, NumPy, Pydantic (Data manipulation, analysis, numerical computation, data validation and serialization).
- Bcrypt, Python dotenv, python-multipart (Password hashing, environment variable, file upload handling).

FRONTEND TECHNOLOGIES

- React, TypeScript (UI components framework and Typed JavaScript Superset).
- Tailwind CSS (Utility-first CSS framework).
- Axios (HTTP client for API calls).
- Reac-hot-toast, react-icons (Toast Notifications and Icon Library).

DEVOPS AND DEPLOYMENT TOOLS

- Docker, PodMan (Containerization of backend and frontend).
- Nginx (Frontend static file serving and reverse proxy)
- Git, GitHub (Code Version Control)
- For Personal Testing the system is deployed on Render (Backend) and Vercel (Frontend).

2.4. HARDWARE AND SOFTWARE REQUIREMENTS

2.4.1 HARDWARE REQUIREMENTS

- Processor: Intel Celeron or higher
- RAM: 512MB or higher
- Storage: 2GB

2.4.2 SOFTWARE REQUIREMENTS

- Operating System: Windows XP / Linux / MacOS
- Web Browser: Chrome / Explorer
- Code Editor: Visual Studio Code
- Node.js and NPM
- Docker, Git (optional)

2.4.3 DEPLOYMENT MACHINE

- CPU: 1 vCPU
- RAM: 512 MB
- Storage: 2GB

CHAPTER 3 : SOFTWARE REQUIREMENTS SPECIFICATION

3.1 INTRODUCTION

Software Requirements Specification (SRS) describes the complete functionality and behavior of the proposed system. It serves as a blueprint for system design and implementation. This chapter identifies the users of the Rake Planning System and defines both functional and non-functional requirements required for the successful development of the system.

3.2 USERS

The Rake Planning System is designed for the following types of users:

3.2.1 Administrator (`admin`)

- Full access to all system modules.
- Can trigger data uploads, force API refreshes.
- Access to all dashboards, GM module, and export functions.

3.2.2 Panel 1 User (`pmgm`)

- Standard operational user role.
- Access to main dashboard, filters, charts, consignee summary, and thickness matrix.
- Can upload files and trigger API data fetch.
- Can export data.

3.2.3 Panel 2 User (`citgm`)

- Department-specific operational access.
- Similar access to Panel 1 with specific data scope.

3.2.4 Viewer (`viewer`)

- Read-only access.
- Can view dashboards and charts.
- Cannot upload data or force API refresh.

3.3 FUNCTIONAL REQUIREMENTS

Functional requirements describe the actions and operations that the system must perform.

3.3.1 USER AUTHENTICATION

- The system shall provide a login interface accepting username and password.
- On successful authentication, a JWT token (valid for 8 hours) shall be issued.
- All API endpoints (except `/login`) shall require a valid JWT token.

3.3.2 DATA UPLOAD

- The system shall allow authenticated users to upload CSV or Excel (.xlsx) files.
- Uploaded files shall be cleansed (column normalization, type conversion, deduplication).
- Cleansing statistics (rows removed, columns normalized) shall be returned.
- Uploaded/cleansed data shall be persisted to disk (CSV cache).

3.3.3 DATA FILTERING

- The system shall support multi-select filters for: Destination, Quality, HT Type, Order Type, Width (WTH), Region, Dispatch Mode and Exclude by Status, Exclude by Grade.
- A pre-configured "Rail Dispatch" preset filter shall be available.
- Filters shall be applied in real time without page reload.

3.3.4 CONSIGNEE SUMMARY

- The system shall generate a consignee-wise summary table including:
- Consignee name, Destination, Loadable Material, Wagon count, Rake count.
- BQT, HT, and N material breakdowns per consignee.
- A TOTAL row shall be appended to the summary.

3.3.5 DESTINATION × THICKNESS MATRIX

- The system shall generate a cross-tabulation of Destination vs. Thickness categories.
- Clicking a cell shall display a consignee drill-down for that destination/thickness combination.

3.3.6 GM MODULE (GRADE MIX ANALYSIS)

- The system shall provide a separate module for comparing two uploaded data files.
- The module shall compute matching combinations based on Quality, HT, Width, Length, Thickness, Trim.
- Results shall be exportable as CSV.

3.3.7 ZCMO MODULE

- The system shall provide ZCMO-specific filtering including:
- Include/exclude by status, Length, Thickness, Trim type.
- UST Level blank filter.

3.4 NON-FUNCTIONAL REQUIREMENTS

3.4.1 PERFORMANCE

- Dashboard KPI calculations for datasets of up to 100,000 rows shall complete in under 3 seconds.
- API response time for filtered queries shall not exceed 5 seconds under normal load.

3.4.2 RELIABILITY

- The system shall persist uploaded/cached data to disk so data survives server restarts.
- On startup, the system shall auto-load the most recently cached dataset.

3.4.3 SECURITY

- All API endpoints shall be protected with JWT authentication.
- Passwords shall never be stored in plain text (bcrypt hashing).
- CORS shall be restricted to configured allowed origins.
- Sensitive configuration (passwords, secret keys) shall be sourced from environment variables, not hardcoded in source code for production deployments.

3.4.4 USABILITY

- The interface shall be responsive and usable at 1280×720 and above.
- Filter selections shall persist across tab switches within a session.
- Loading indicators shall be shown during data fetch and calculation operations.

3.4.5 SCALABILITY

- The backend shall be stateless (data stored in server-side file cache) to allow horizontal scaling behind a load balancer.
- The Docker-based deployment shall enable rapid scaling.

3.4.6 MAINTAINABILITY

- Backend logic shall be modularized into `core/` sub-packages (calculations, filters, data_cleansing, utils, constants).
- Frontend components shall be organized by feature.
- Configuration values (wagon capacity, rake size, API URL) shall be centralized in `config.py`.

3.5 SUMMARY

This chapter defined the software requirements of the Rake Planning System. It identified different system users and detailed the functional and non-functional requirements necessary for system development. These requirements ensure that the system is secure, reliable, scalable, and user-friendly.

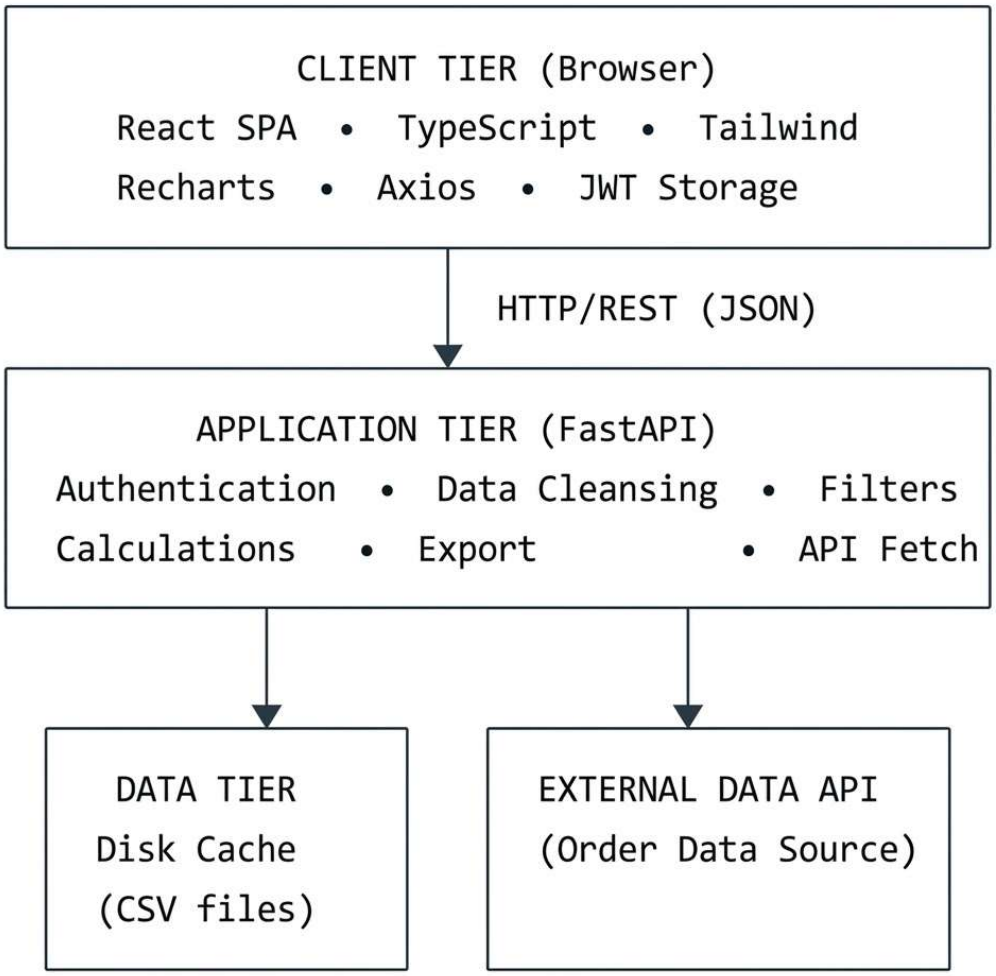
CHAPTER 4: SYSTEM DESIGN

4.1 INTRODUCTION

System design defines the overall architecture of the proposed system and explains how different components interact with each other. This chapter presents the system perspective, various diagrams such as context diagram, use case diagram, sequence diagram, collaboration diagram, activity diagram, and database design of the Rake Planning System.

4.2 SYSTEM PERSPECTIVE

The Rake Planning System follows a three-tier architecture:

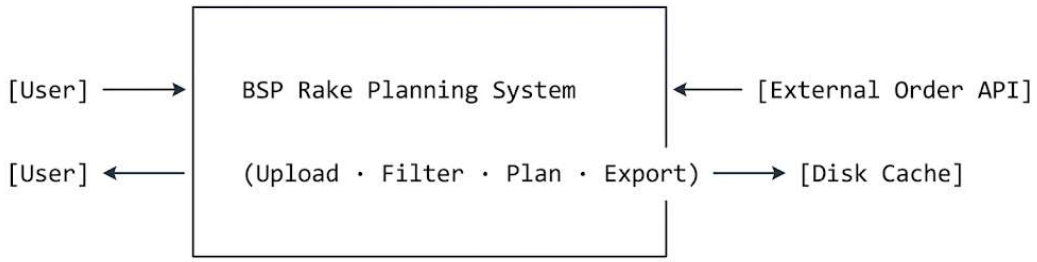


Client Tier: The React single-page application runs in the user's browser. It communicates exclusively with the FastAPI backend via RESTful HTTP calls, sending the JWT token in the `Authorization` header.

Application Tier: The FastAPI backend handles authentication, data ingestion, cleansing, filtering, calculations, and export generation. All business logic is implemented in Python using Pandas/NumPy.

Data Tier: Processed data is persisted as CSV files on the server's filesystem. No relational database is required — the data model is flat tabular data derived from steel order records.

4.3 CONTEXT DIAGRAM (DFD — LEVEL 0)



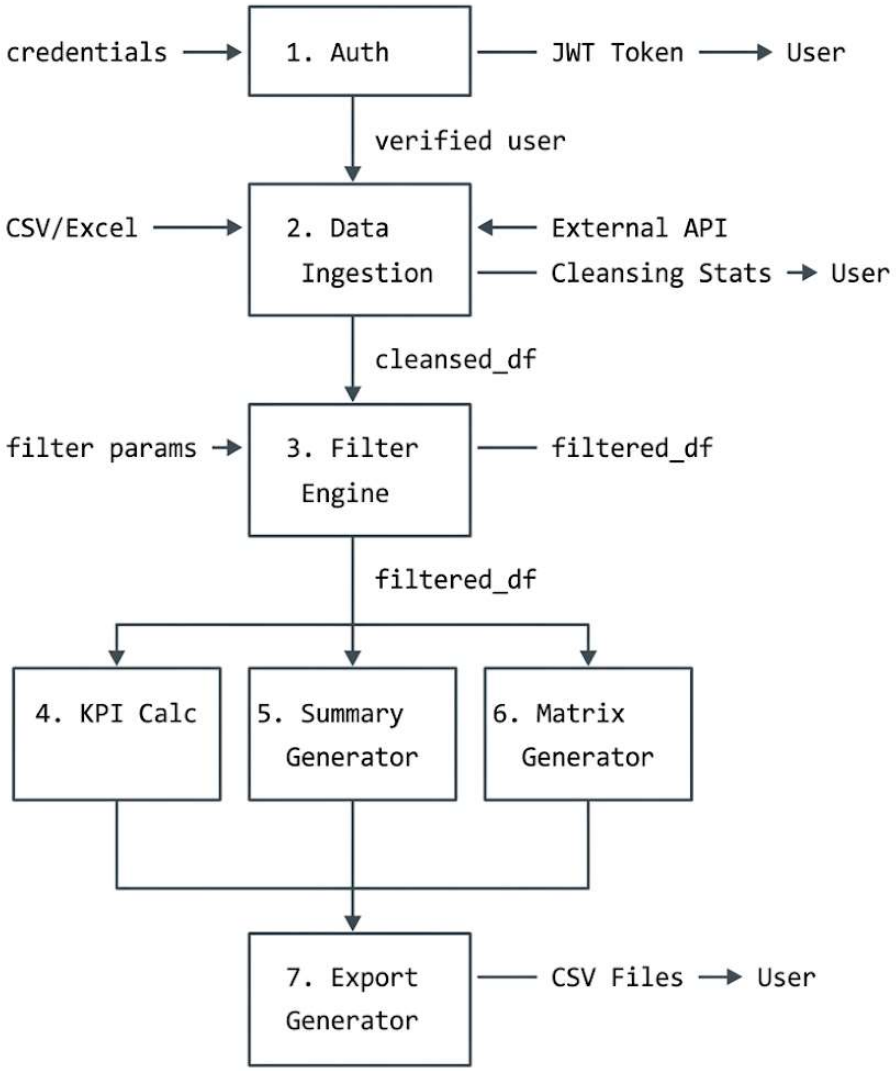
4.3.1 EXTERNAL ENTITIES:

- **User —** Authenticated personnel (Admin, PMGM, CITGM, Viewer) accessing the system via browser.
- **External Order API —** Upstream ERP/data system providing raw order data in CSV/JSON format.
- **Disk Cache —** Server-side file storage for persisting cleansed data.

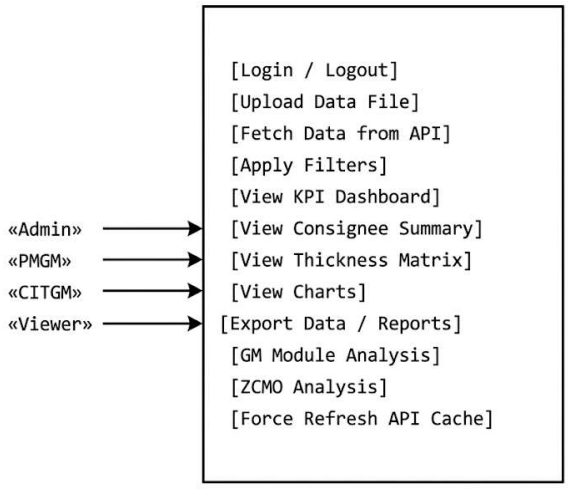
4.3.2 DATA FLOWS:

- **User → System**: Login credentials, file uploads, filter selections, export requests.
- **System → User:** JWT token, KPI metrics, summary tables, chart data, downloaded files.
- **External API → System:** Raw order records.
- **System → Disk Cache:** Cleansed CSV data (write/read).

4.4 DFD — LEVEL 1



4.4.1 USE CASE DIAGRAM



4.5 SEQUENCE DIAGRAMS

4.5.1 AUTHENTICATION (USER LOGIN)

- User enters credentials (username, password).
    - **Browser (React) sends POST /login to FastAPI.**
    - **FastAPI:**
- Verifies password using bcrypt hash.
- Generates JWT token.
- FastAPI returns 200 OK with { token, role }.
- Browser stores JWT in memory (or local storage).
- User is redirected to the Dashboard.

4.5.2 UPLOAD, FILTER AND DASHBOARD

FILE UPLOAD

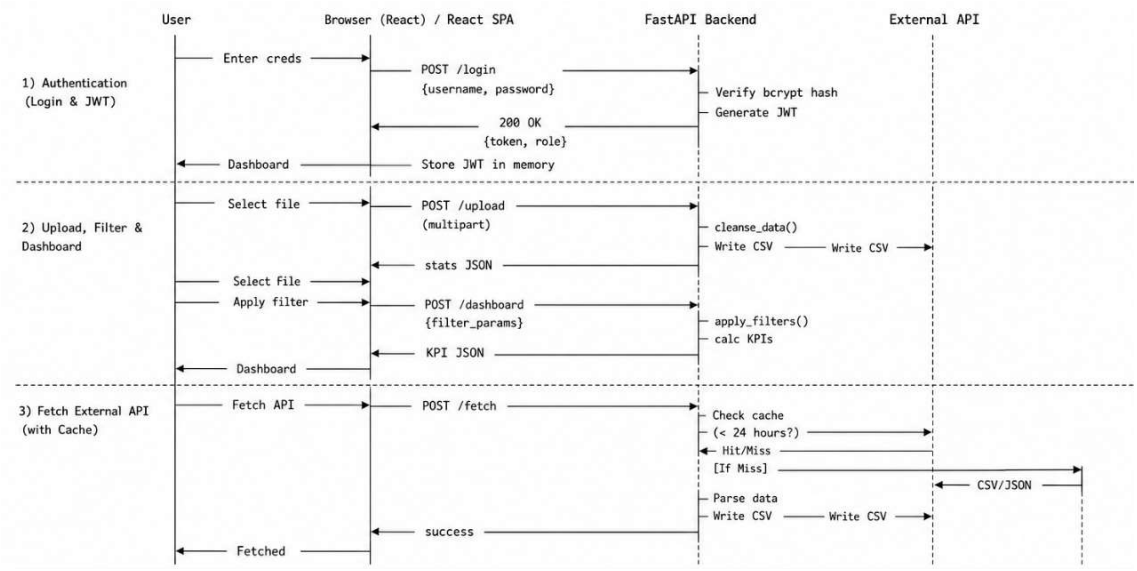
- User selects a file.
- Browser sends POST /upload (multipart) to FastAPI.
- FastAPI: Cleanses data (cleanse_data()).
- FastAPI: Writes processed data as CSV to disk cache.
- FastAPI returns stats JSON to the browser.

APPLY FILTERS & VIEW DASHBOARD

- User selects file & applies filters.
- Browser sends `POST /dashboard` with `{ filter_params }`.
- FastAPI: Applies filters (`apply_filters()`).
- FastAPI: Calculates KPIs.
- FastAPI returns `KPI JSON`.
- Browser renders Dashboard with updated data.

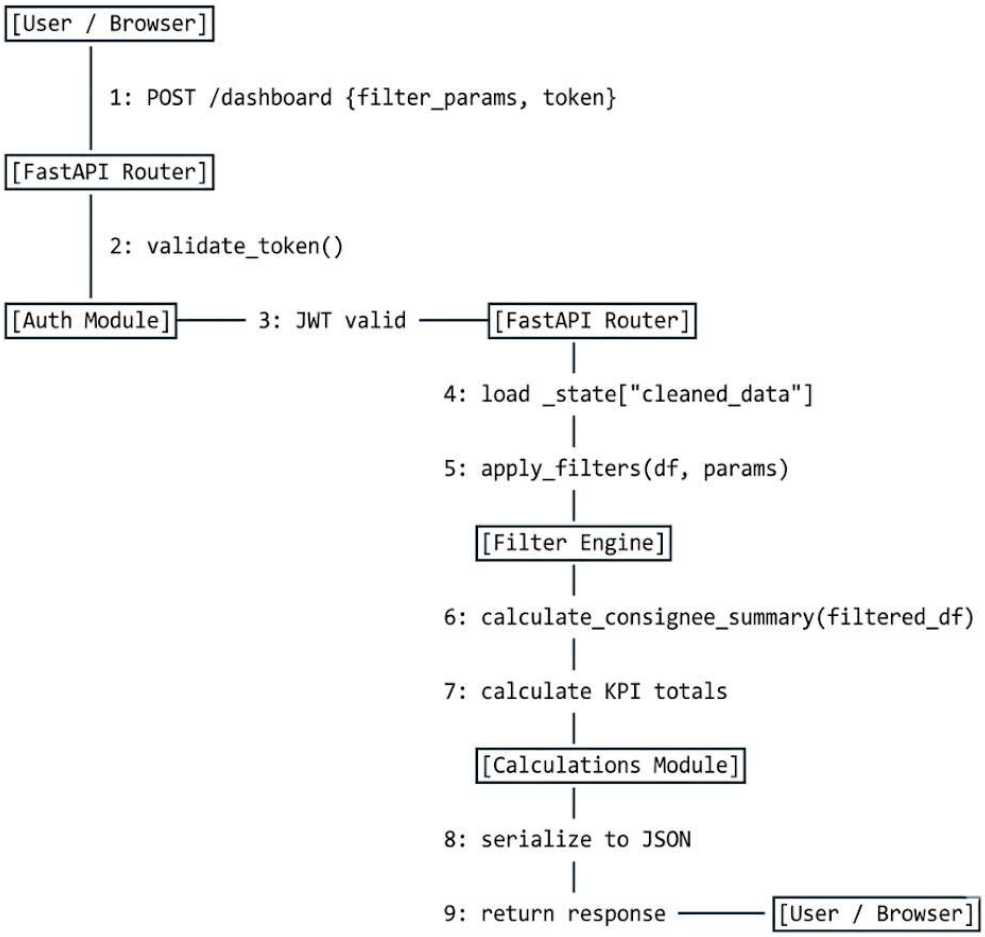
4.5.3 DATA FETCH AND CACHE

- User uploads CSV, Excel data
- Browser sends POST /fetch to FastAPI.
- FastAPI: Checks disk cache (valid if < 24 hours).
- If **cache hit**: Return cached data immediately.
- If **cache miss**: Receive data (CSV/JSON).
- Parse data. Store as CSV in disk cache.
- FastAPI returns success response.

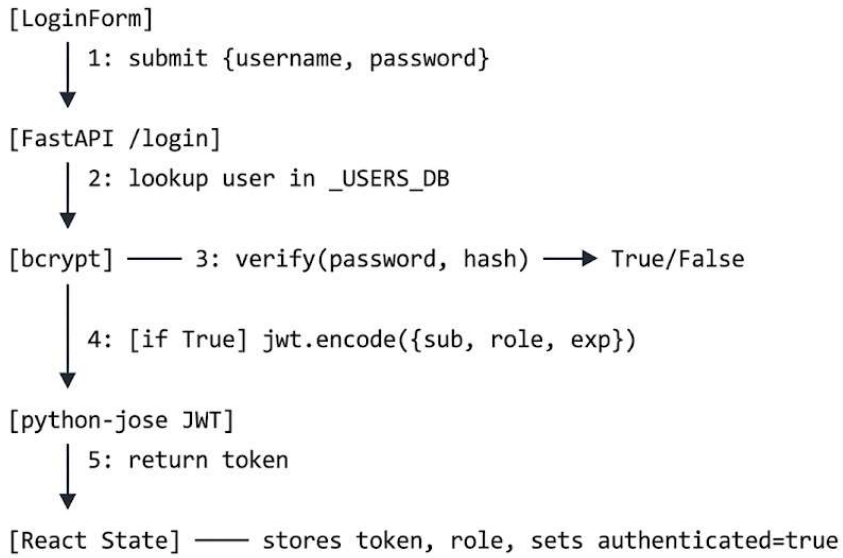


4.6 COLLABORATION DIAGRAMS

4.6.1 DASHBOARD DATA FLOW

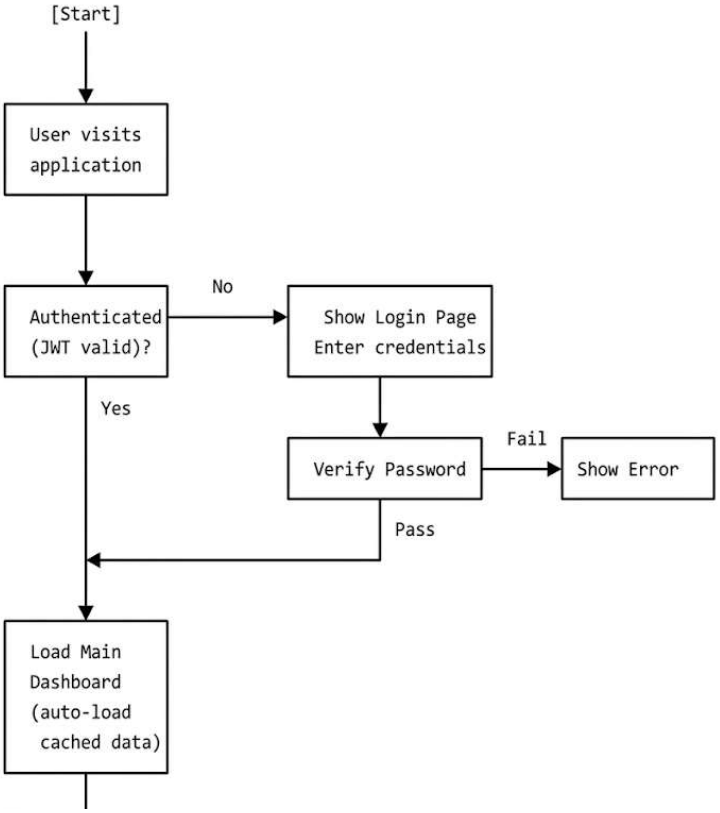


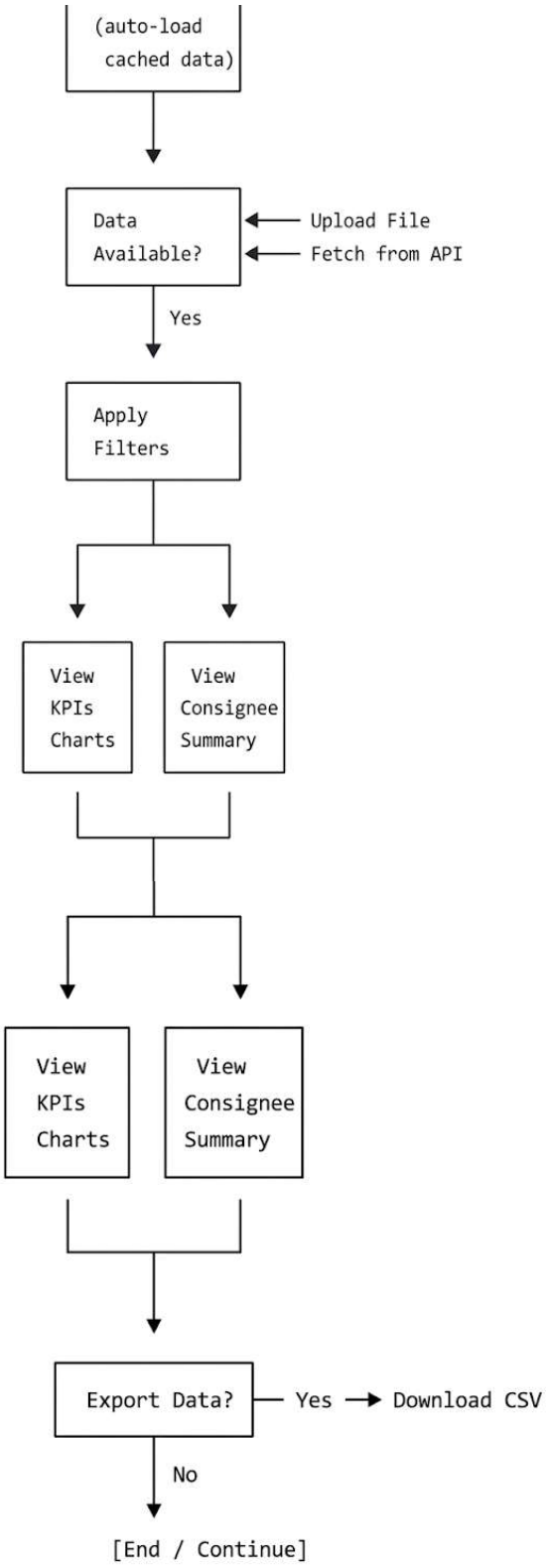
4.6.2 AUTHENTICATION



4.7 ACTIVITY DIAGRAM

4.7.1 MAIN APPLICATION FLOW





The above flow diagram complete flow of Main Application from starting to end.

4.8 DATABASE DESIGN

4.8.1 OVERVIEW

The BSP Rake Planning System uses a **flat-file data store** rather than a relational database. The core data is tabular (order records), sourced from CSV/Excel uploads or API responses. After cleansing, data is persisted as CSV files on the server's filesystem.

This approach is appropriate because:

- The data is read-heavy (analyzed, not mutated).
- The upstream data source (ERP system) is the system of record.
- Pandas DataFrames provide all required query/aggregation capabilities in memory.

4.8.2 CORE DATA SCHEMA (ORDER RECORD)

Column	Type	Description
CONSIGNEE	String	Customer/consignee name
CONSIGNEE CD	String	Consignee code
DESTINATION	String	Destination location
MR	Float	Material Ready (MT)
BFD	Float	Bill For Dispatch (MT)
QUALITY	String	Steel quality (MILD, BOILER, HIGH TENSILE, etc.)
HT	String	Heat Treatment type (N, NR, UN, etc.)
WTH	Float	Width (mm)
LEN	Float	Length (mm)
ORDER TYPE	String	Order classification
REGION	String	Geographic region
DISPATCH MODE	String	Mode of dispatch (RAIL, ROAD, etc.)
STATUS	String	Order status
GRADE	String	Steel grade code
UP	Float	Unit Price
AWAIT TEST	Float	Material awaiting test (MT)
LD Date	Date	Last dispatch date
TRIM	String	Trim type (TRIM / UNTRIM)

4.8.3 DERIVED / CALCULATED COLUMNS

Column	Formula	Description
Loadable_Material	MILD: MR + AWAIT_TEST; Others: MR	Total loadable MT
BQT_Material	Loadable_Material where QUALITY = BOILER	Boiler quality loadable material
HT_Material	Loadable_Material where QUALITY = HIGH TENSILE	HT loadable material
N_Material	Loadable_Material where HT = N AND QUALITY ≠ BOILER	Normalized material
Wagons	Loadable_Material / 60 (WAGON_CAPACITY_MT)	Wagons required
Rakes	Wagons / 43 (RAKE_SIZE_WAGONS)	Rakes required

CHAPTER 5 : IMPLEMENTATION

5.1 INTRODUCTION

Implementation is the phase where the system design is converted into a working software product. In this stage, the Rake Planning System, was developed using Python and React technology and modern web development tools. The implementation includes Data Processing, Frontend UI design, and deployment on a live environment.

The main aim of this chapter is to explain how the system was implemented and to describe the working of the application using screenshots.

5.2 IMPLEMENTATION DETAILS

The Rake Planning System is implemented using a centralized architecture. It consists of two main major modules.

1. **Backend Module (API / Data Processing).**
2. **Frontend Module (User Interface Layer).**

5.3 BACKEND IMPLEMENTATION

5.3.1 DATA CLEANSING PIPELINE

- Read CSV/Excel into a Pandas DataFrame.
- Normalize column names (strip whitespace, upper-case).
- Map variant column names to canonical names (e.g., `DEST` → `DESTINATION`).
- Convert numeric columns (THK, WTH, LEN, MR, BFD, etc.) to float.
- Fill NaN values with appropriate defaults (0 for numerics, '' for strings).
- Remove duplicate records.
- Return cleansed DataFrame and statistics dict.

5.3.2 FILTER ENGINE (CORE/FILTERS.PY)

- Accepts a DataFrame and a `FilterParams` object.
- Sequentially applies each filter criterion using boolean indexing.
- Supports both include-list filters (e.g., destinations) and exclude-list filters (e.g., exclude_status). Returns a filtered DataFrame view (no data mutation).

5.3.3 KPI CALCULATIONS

- `calculate_consignee_summary()`: Groups filtered data by consignee, aggregates Loadable Material, BQT, HT, N material, calculates wagon and rake counts.
- `calculate_destination_thickness_matrix()`: Pivots filtered data into a Destination × Thickness cross-tab using predefined thickness buckets from `constants.py`.
- `get_consignee_details_by_destination()`: Returns individual order details for a given destination and thickness range (used for drill-down).
- `add_total_row()`: Appends an aggregated TOTAL row to summary DataFrames.

5.3.4 AUTHENTICATION

- `POST /login`: Validates credentials against `_USERS_DB` (bcrypt hash comparison), returns signed JWT with `sub` (username) and `role` claims adn 8-hour expiry.
- `_require_auth(token)`: Dependency-injected function that decodes and validates the JWT on every protected endpoint.

5.4 FRONTEND ARCHITECTURE (REACT)

- **Authentication Context**: Stores JWT token and role in React state; persists to `sessionStorage`.
- **API Service**: Axios instance with base URL and auto-injected `Authorization` header.
- **Dashboard Component**: Fetches KPIs and consignee summary on filter change.
- **Chart Component**: Renders selected chart type from Recharts using consignee summary data.
- **Filter Panel**: Multi-select dropdowns populated from `/filter-options` endpoint.
- **Thickness Matrix**: Table component with click-to-drill-down functionality.

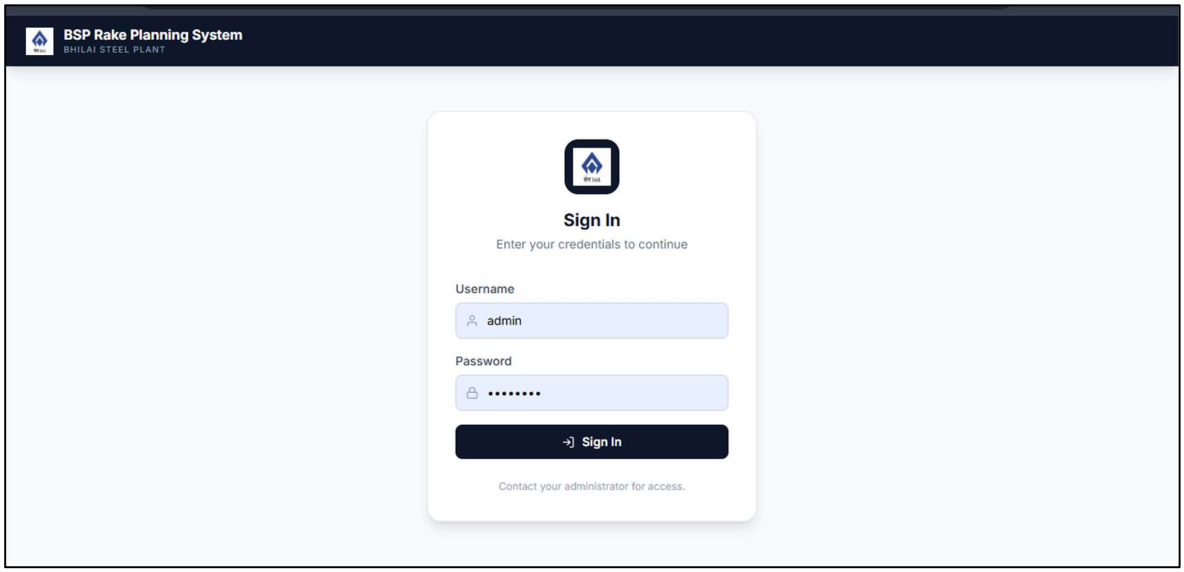
5.4.1 API ENDPOINTS (KEY SELECTION)

Method	Endpoint	Description
POST	/login	Authenticate, receive JWT
POST	/upload	Upload CSV/Excel data file
POST	/fetch-api	Fetch data from configured external API
GET	/filter-options	Get available filter values

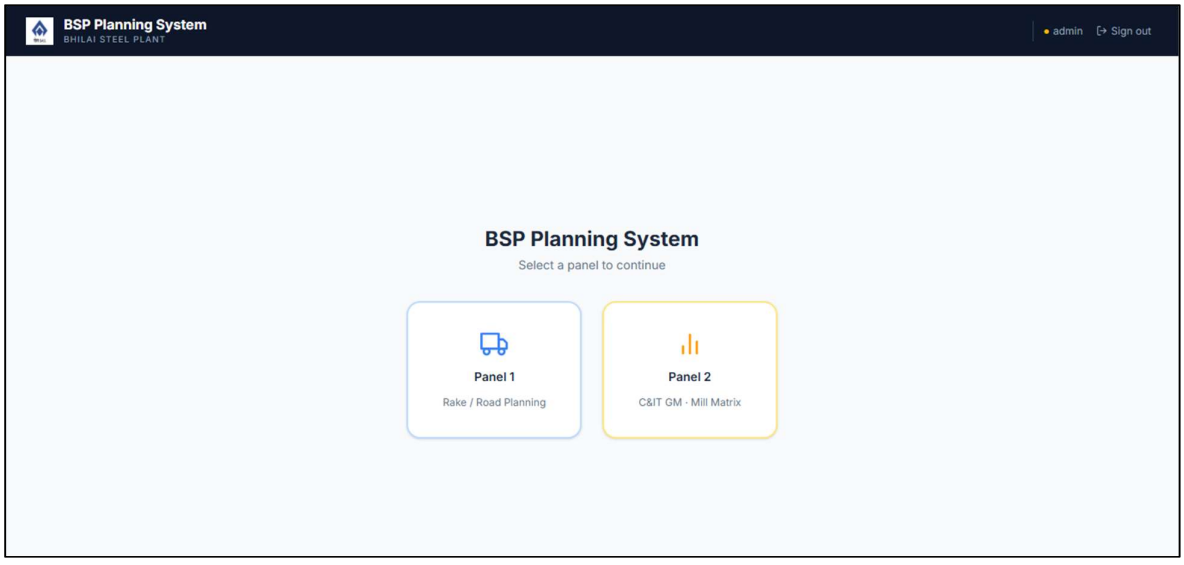
POST	/dashboard	Get KPIs (loadable material, wagons, rakes)
POST	/consignee-summary	Get consignee summary table
POST	/thickness-matrix	Get destination × thickness matrix
POST	/consignee-details	Get drill-down details by destination/thickness
POST	/gm/upload	Upload GM analysis file 1 or 2
POST	/gm/matching-combos	Get matching combinations for GM analysis
GET	/health	Health check endpoint

5.5 SCREENSHOTS

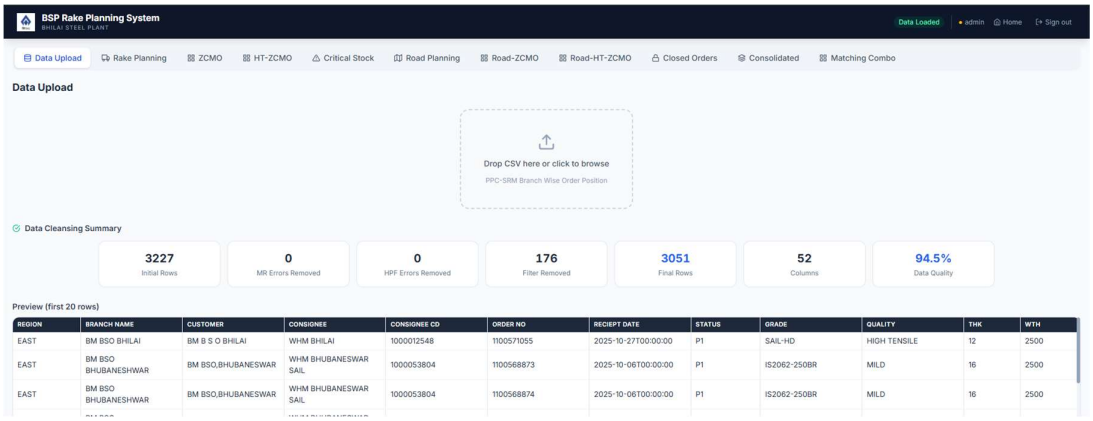
SCREEN 1: LOGIN PAGE



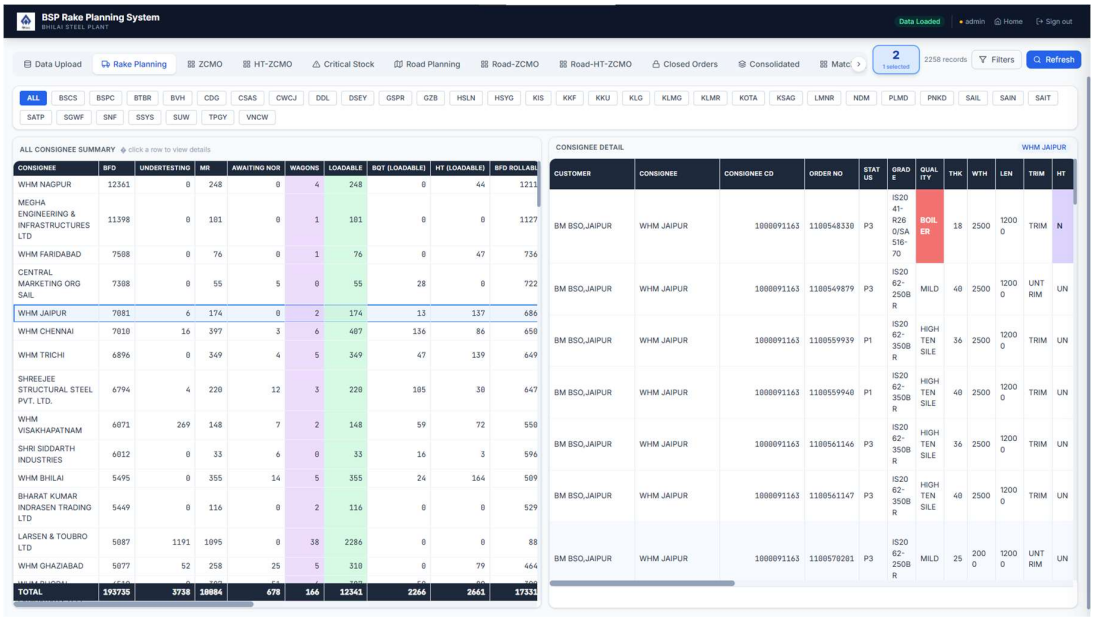
SCREEN 2: PANEL SELECTOR

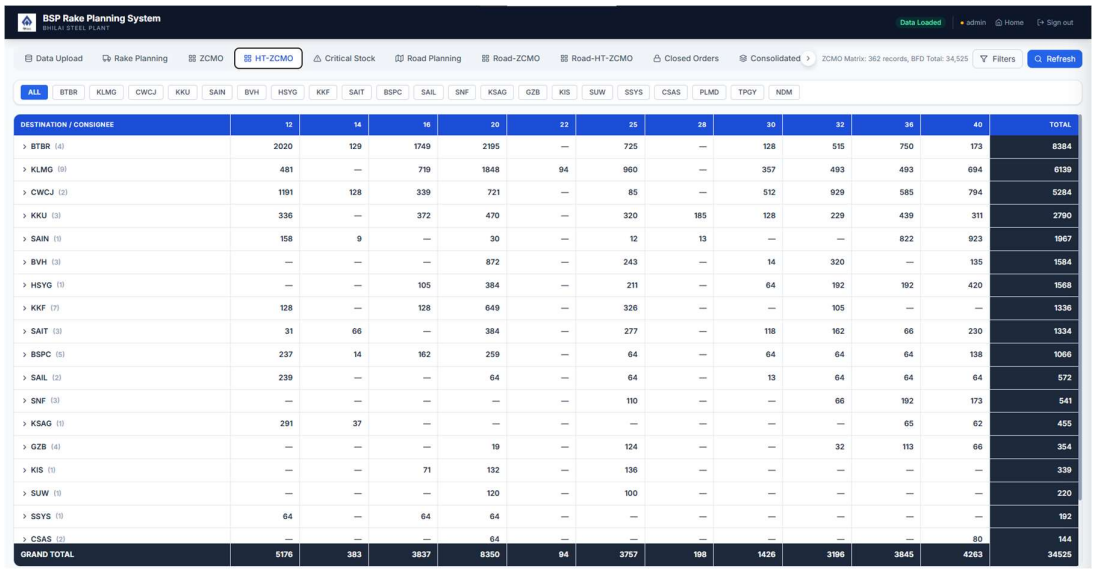


SCREEN 3: RAKE PLANNING - DATA UPLOAD – PANEL - 1

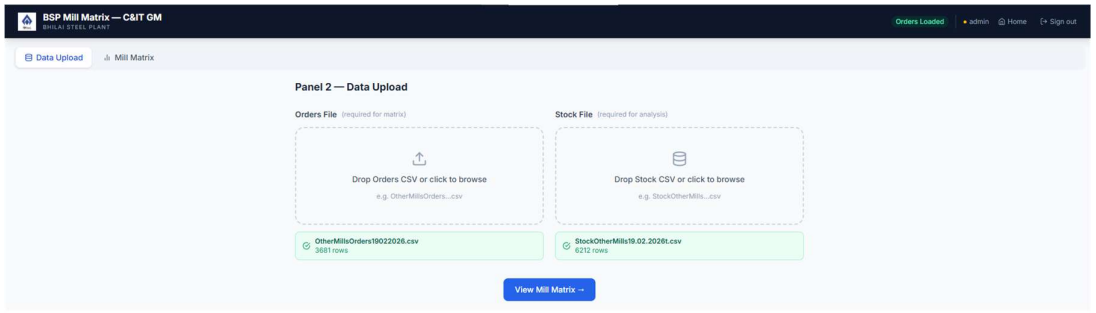


SCREEN 4: RAKE PLANNING - CONSIGEE SUMMARY

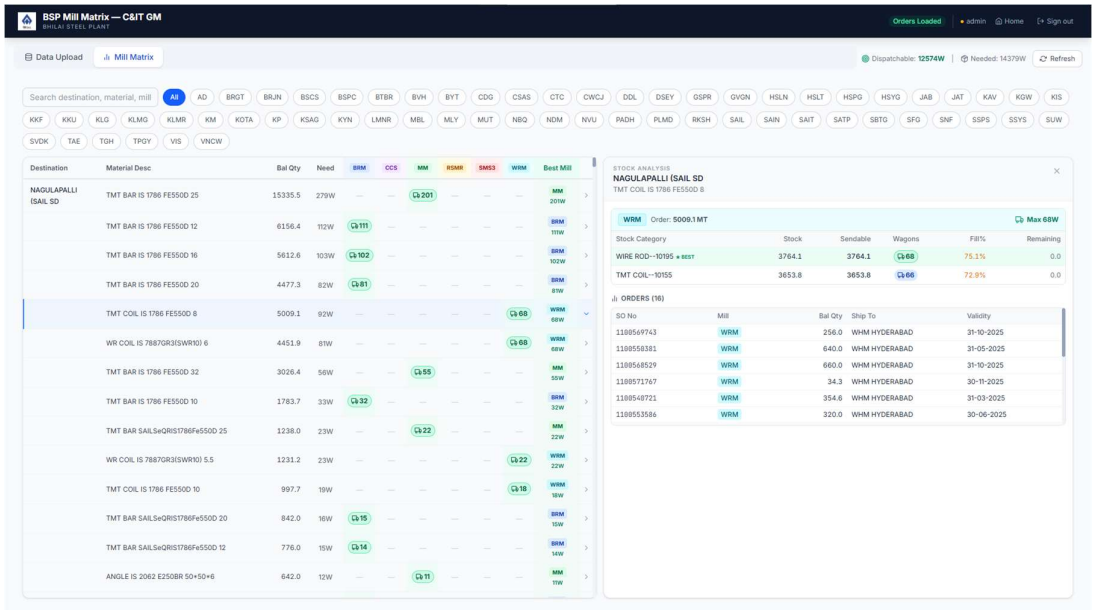




SCREEN 5: GM MODULE – PANEL 2



SCREEN 6 : GM MODULE – MILL MATRIX



CHAPTER 6 : SOFTWARE TESTING

6.1 INTRODUCTION

Software testing is a critical phase of software development life cycle that ensures the developed system works correctly and meets the specified requirements. Testing is performed to identify errors, verify functionality, and ensure that the final product is reliable, secure, and user-friendly.

6.2 OBJECTIVES OF TESTING

The main objectives of testing in this project are:

- To verify that all modules work according to requirements.
- To ensure all data must be accurate and consistent.
- To check the system is reliable and secure.
- To ensure proper UI functioning without errors.

6.3 TYPES OF TESTING PERFORMED

- **Unit Testing:** Core Python functions (calculations, filters, data cleansing).
- **Integration Testing:** API endpoint testing using Postman and HTTP client tools.
- **User Acceptance Testing:** End-to-end testing with real order data files by domain users.
- **User Interface Testing:** Proper rendering of pages, button clicks, navigation links.
- **Security Testing:** Only authorized users can gain access.
- **Performance Testing:** Load testing with large datasets (50,000+ rows).

6.4 TEST PLAN

The testing process followed the below steps:

- Define test scenarios and expected output
- Perform unit testing for individual modules
- Identify bugs and correct them
- Re-test the system after fixing issues
- Validate final working on deployed version

6.5 TEST CASES

Below are the test cases performed on the Rake Planning System:

MODULE: AUTHENTICATION

TC No.	Test Case	Input	Expected Output	Status
TC-01	Valid login	username: admin, password: admin123	200 OK, JWT token returned	Pass
TC-02	Invalid password	username: admin, password: wrongpass	401 Unauthorized	Pass
TC-03	Unknown user	username: nobody, password: x	401 Unauthorized	Pass

MODULE: DATA UPLOAD

TC No.	Test Case	Input	Expected Output	Status
TC-06	Valid CSV, Excel upload	Well-formed order CSV, .xlsx order file	200 OK, cleansing stats, data loaded	Pass
TC-08	Invalid file type / empty file	.pdf file, 0 rows CSV	400 Bad Request / Warning returned	Pass

MODULE: FILTERING

TC No.	Test Case	Input	Expected Output	Status
TC-11	Filter by destination	destinations: ["MUMBAI"]	Only MUMBAI rows returned	Pass
TC-12	Filter by quality	qualities: ["BOILER"]	Only BOILER quality rows	Pass
TC-13	Exclude by status	exclude_status:["PENDING"]	No PENDING status rows	Pass
TC-14	Multiple filters combined	destinations + qualities + HT	Intersection of all conditions	Pass

MODULE: PERFORMANCE

Dataset Size	Operation	Response Time	Status
10,000 rows	Dashboard KPI calculation	< 0.5 seconds	Pass
50,000 rows	Dashboard KPI calculation	~1.2 seconds	Pass
100,000 rows	Dashboard KPI calculation	~2.8 seconds	Pass
50,000 rows	Thickness matrix generation	~1.5 seconds	Pass

CHAPTER 7: RESULTS AND DISCUSSION

7.1 INTRODUCTION

This chapter presents the final results obtained after implementing and testing the Rake Planning System. It discusses how the project fulfills its objectives and provides the observed outputs in terms of system performance, user interaction, data upload and overall working of the application. The results are discussed based on functional implementation and user experience.

7.2 RESULTS

The deployed BSP Rake Planning System successfully achieves all defined objectives:

- **Automated KPI Calculation**: Wagon and rake requirements are now computed automatically in sub-second time for typical datasets, eliminating error-prone manual formula calculations.
- **Centralized Data Management**: A single web application replaces multiple Excel files shared across departments, eliminating version control issues.
- **Role-Based Access**: Four distinct roles (admin, pmgm, citgm, viewer) ensure each user sees only what is relevant to their function, improving data governance.
- **Multi-dimensional Analysis**: Users can simultaneously filter by 8+ dimensions and view results across KPI cards, consignee summary, thickness matrix, and 8 chart types — a significant improvement over pivot table-based Excel analysis.
- **API Integration with Caching**: Daily automatic refresh from the upstream order API ensures data freshness without manual file downloads, while caching prevents unnecessary API load.
- **Export Capability**: One-click CSV export of filtered data, summaries, and matrices replaces manual copy-paste workflows.

7.3 USER FEEDBACK (UAT)

- End users noted a significant reduction in time to prepare dispatch summaries: from approximately 45 minutes (Power BI) to under 5 minutes (web application).
- Management appreciated the always-accessible browser-based interface that eliminated the need to transfer files.
- The GM Module for grade mix analysis was highlighted as a key value-add not previously available in the manual system.

CHAPTER 8: CONCLUSION / SUMMARY

8.1 INTRODUCTION

This chapter presents the conclusion of the **Rake Planning System** project and highlights the future enhancements that can be implemented.

8.2 CONCLUSION

The **BSP Rake Planning System** is a purpose-built, full-stack web application designed to modernize and automate the rake dispatch planning operations at Bhilai Steel Plant's Plate Dispatch Section.

The system replaces a manual, Excel-based workflow with a centralized, browser-accessible dashboard that provides:

- Secure, role-based authentication.
- Automated data ingestion via file upload or API integration.
- Real-time, multi-dimensional filtering.
- Automated KPI and rake planning calculations.
- Interactive data visualization across 8 chart types.
- One-click export of all analysis outputs.

By leveraging modern open-source technologies — FastAPI for a high-performance Python backend, React with TypeScript for a responsive frontend, and Docker for production-ready containerization — the system is robust, maintainable, and deployable on any cloud platform at negligible infrastructure cost.

The project demonstrates how targeted digital transformation of operational workflows — even without large-scale ERP changes — can deliver immediate, measurable productivity gains for industrial operations.

CHAPTER 9 : FUTURE ENHANCEMENTS

9.1 INTRODUCTION

Future enhancements describe additional improvements and features that can be implemented after the completion of the current system. Although the **Rake Planning System** fulfills the basic objectives of BSP rake management workflow needs, some advanced functionalities can be added to make the application more scalable, efficient, and suitable for real-world adoption.

9.2 The following enhancements are identified for future development cycles:

9.2.1 DYNAMIC EMPLOYEE MANAGEMENT

Implement an Admin UI for creating, editing, and deactivating employee accounts, backed by a lightweight SQLite or PostgreSQL database for user records.

9.2.2 RAKE SCHEDULE BUILDER (INTERACTIVE)

Add a drag-and-drop rake builder where dispatchers can manually assign consignees to specific wagons within a rake and generate a printable rake schedule document.

9.2.3 HISTORICAL DATA AND TREND ANALYSIS

Store historical snapshots of daily order position data to enable trend analysis (material available over time, dispatch velocity, rake turnaround time).

9.2.4 EMAIL AND NOTIFICATION ALERTS

Automated daily email reports summarizing the current order position and rake plan to configured distribution lists.

9.2.5 ERP WRITE-BACK INTEGRATION

Bidirectional ERP integration — mark orders as dispatched, update BFD status — directly from the rake planning dashboard.

9.2.6 REAL-TIME COLLABORATION

WebSocket-based real-time updates so multiple users see live filter changes and data updates without manual refresh.

9.2.7 PREDICTIVE ANALYTICS

Integrate a simple ML model to predict expected dispatch dates based on historical patterns and current order position.

CHAPTER 10 : BIBLIOGRAPHY / REFERENCES

The following references were used during the study, design, and implementation of the **Rake Planning System** project:

- FastAPI Documentation — Sebastián Ramírez. **FastAPI — Modern, Fast Web Framework for Building APIs with Python**. https://fastapi.tiangolo.com/
- React Documentation — Meta (Facebook). *React* — **A JavaScript Library for Building User Interfaces**. https://react.dev/
- Pandas Documentation — The Pandas Development Team. **pandas: Powerful Python Data Structures** https://pandas.pydata.org/docs/
- NumPy Documentation — The NumPy Community. **NumPy — The Fundamental Package for Scientific Computing with Python.** https://numpy.org/doc/
- Recharts Documentation — Recharts Group. **Recharts — A Composable Charting Library Built on React Components** https://recharts.org/
- Pydantic Documentation — Samuel Colvin et al. **Pydantic — Data Validation Using Python Type Hints.** https://docs.pydantic.dev/
- Docker Documentation — Docker Inc. **Docker: Containerization Platform.** https://docs.docker.com/
- Vite Documentation — Evan You et al. **Vite — Next Generation Frontend Tooling.** https://vitejs.dev/
- Tailwind CSS Documentation — Tailwind Labs. **Tailwind CSS — A Utility-First CSS Framework.** https://tailwindcss.com/docs/
- JWT (JSON Web Tokens) — RFC 7519. **JSON Web Token (JWT).** https://datatracker.ietf.org/doc/html/rfc7519
- **SAIL / Bhilai Steel Plant — Steel Authority of India Limited. About BSP.** https://www.sail.co.in/

CHAPTER 11 : APPENDIX A : USER MANUAL

11.1 INTRODUCTION

This user manual provides step-by-step guidance for using the Rake Planning System. This platform allows BSP managers to plan wagons count and order, stock summary.

11.2 SYSTEM REQURIMENTS

To use the Rake Planning System successfully, users must have:

11.2.1 HARDWARE REQUIREMENTS

- Laptop/PC/Mobile Device
- Minimum 4 GB RAM (recommended)
- Stable internet connection

11.2.2 SOFTWARE REQUIREMENTS

- Web Browser: Google Chrome / Firefox / Edge
- MetaMask Wallet Extension (Installed and configured)
- Crypto test funds (if running on testnet)

11.3 HOW TO ACCESS THE APPLICATION

STEP 1: OPEN THE WEBSITE

- Open any supported browser.
- Enter the URL: https://bsp-rake.vercel.app
- Upload the CSV, that’s it. Now navigate to desired tab you wanted to see.

11.4 SUMMARY

This user manual described how to use the Rake Planning System including wallet connection, campaign browsing, campaign creation, funding process, and withdrawal process.

CHAPTER 12: APPENDIX B : SOURCE CODE

12.1 SOURCE CODE

12.1.1 GITHUB REPOSITORY:

https://github.com/girdharagrawalbro/BSP-Rake

12.2 DEPLOYMENT STEPS

12.2.1 INSTALL DEPENDENCIES BACKEND

pip install –r requirements.txt

This will install all required dependencies

12.2.2 RUN THE BACKEND

python api.py

12.2.1 INSTALL DEPENDENCIES FRONTEND

npm install

12.2.2 RUN THE FRONTEND

npm run dev

12.3 PROJECT LINKS

12.3.1 LIVE DEPLOYED WEBSITE

https://bsp-rake.vercel.app

12.6 SUMMARY

Appendix B contained source code, deployment steps, and important code overview. These appendices support the implementation and validate the working of the project.